

ALGO FINAL KONUSLAR

Greedy

Divide & Cong.

Hashing

- Open Addressing
= linear probing
= Double hashing
= Universal hashing
- Separate Chaining

DP $\Rightarrow$ bir Deneyim!

- Knapsack
- Edit Distance
- Longest Common Subsequence

Graph Traversal

- DFS
= stack
= Recursion
- BFS
= queue
- Topological Sort
- (source, vertex) yolunu
- connected comp say
- (V_1, V_2) en yakın adim.
- adj matrix ile de yapılabilir

Backtracking $\Rightarrow$ DFS ile

- N Queen Problem
- Hamiltonian Circuit

Branch & Bound $\Rightarrow$ BFS ile

- Job scheduling
- Knapsack

- Princeton soruları bul.
- LeetCode D&A of A soruları çöz
- + insertion sort yaz.
- Decimal to Binary conversion yaz. Recursive.
- Recurrence relation yaz.

+ Selection Sort yaz.

+ Merge Sort yaz.

- stable sort yaz.

+ Quick sort yaz.

- iterative quick sort

+ Fibonacci - DP.

+ Binomial Coeff. - DP

+ Knapsack - DP

+ coin exchange - DP

+ Edit Dist. - DP

+ Longest Common Subsequence - DP

- Partitioning of a set that 2 sets are containing same sum of nums.

+ DFS ~~iterative~~ recursive

+ Source-Target Arası bulunur? DFS

- Graph Traversal Soruları Çikar

+ Connected Component Sayısı - DFS

+ DFS - non recursive

+ BFS

+ Source-Target arası en kısa yol.

- coin-row problem - DP

+ Topological Sort - DFS

+ N-Queen problem - Backtracking

+ Hamiltonian circuit prob. - Backtracking

- Job Scheduling ^{assignment prob.} - Branch & Bound

- Knapsack - Branch and Bound

- Shortest path - Longest path.

- Activity Selection - Greedy

- Prim's ALGO - Greedy

- Kruskal ALGO - Greedy

- Dijkstra Shortest path.

- Subset Sum problem

- ALL HASH VERSIONS.

①

ALGO

13.10.21

efficiency $\rightarrow$ time
 $\rightarrow$ yes

Execution times

① System dependent effects
 sys OS, network, other apps
 hardware CPU, memory, cache
 - software = compiler, interpreter.

② System independent effects
 - Algorithm complexity
 - Input $\rightarrow$ N & length.

Assumptions

- ① Each single operation (+, -, *, $\div$, =, if) takes exactly one time step
- ② Loops and functions are not ^{considered as} single operations.
- ③ Each memory access takes exactly one time step. (disk, cache, etc)

Example

```

for (i=0; i<n; i++)  $\rightarrow$  i=0 ... n-1  $\rightarrow$  n  $\Rightarrow$  n+1 control steps
{
  for (j=0; j<n; j++)  $\Rightarrow$   $\downarrow$  n * (n+1)  $\uparrow$  j  $\Rightarrow$   $n^2 + n$ 
  {
    i++;  $\rightarrow$   $n^2$ 
    i++;  $\rightarrow$   $n^2$ 
  }
}

```

$$\frac{+}{3n^2 + 2n + 1}$$

(2)

Algo

13.10.21

$n \rightarrow \infty$ problem in terms of n , big $\rightarrow$ independent

① Discard constant terms $3n^2 + 2n + 1 \rightarrow 3n^2 + 2n$

② Discard coefficients $\rightarrow 3n^2 + 2n \rightarrow n^2 + n$

③ Pick highest order term. $\rightarrow n^2 + n \rightarrow n^2 //$

* Komarilik hesabını "n" e bağlı bələli dəyərləndirərək
yaparık. Bu base operation dir. örülte for for ki-

Order of Growth; growth rate of algorithm - $n \rightarrow \infty$

Asymptotic Analysis $n \rightarrow \infty$ hesab yəqarır

Asymptotic Notations

Big-Oh
 $O(n)$

Big Theta
 $\Theta(n)$

Big-Omega
 $\Omega(n)$

Big-Oh

Asymptotic upper Bound $\Rightarrow$

$f(n) \rightarrow$ given functions

there exists constant $c > 0$ and $n_0 \geq 0$

$0 \leq f(n) \leq c \cdot g(n)$ for $n \geq n_0$

$g(n)$ is asymptotic upper bound.

$f(n) = 3n^2 + 4n + 5 \rightarrow$ big highest bit bura n^2 olub $3n^2$ olub $3n^2$ olub $3n^2$ olub $3n^2$ olub

$3n^2 + 4n + 5 \leq c \cdot n^2$

$n \geq n_0$

(3)

Algo

13.10.21

Big-O'nun exagrement

$$f(n) = 10n + 30$$

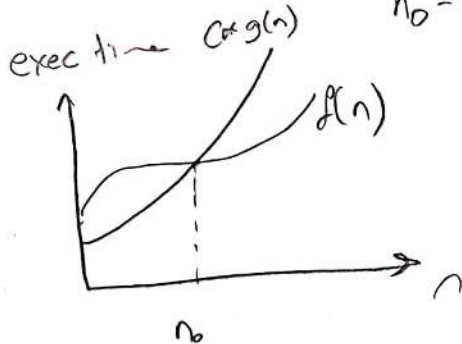
$$f(n) \in O(g(n))$$

$$10n + 30 \leq c \cdot \overbrace{g(n)}^2$$

$$30 \leq (c-10)n$$

$$c=40, n_0=1 \Rightarrow 30 \leq 30 //$$

$$n_0=2 \Rightarrow 30 \leq 60.$$


 $c=40, n_0=1$ sağlanıyor.

$$f(n) \in O(g(n))$$

$$O(n) //$$

$$10n + 30 \in O(n) //$$

Ex2 $f(n) = n^2 + n$

$f(n) \in O(n^2) ?$ $\rightarrow g(n)$

$$n^2 + n \leq c \cdot g(n)$$

$$n^2 + n \leq c \cdot n^2$$

$$0 \leq (c-1)n^2 - n$$

$$0 \leq ((c-1)n - 1)n$$

$$n_0=3, c=5 \checkmark$$

$$n_0=4 \checkmark$$

 $c=5$ ve $n_0=3$ için
 $n \geq n_0$ için
 her "n" sağlanıyor.

$$f(n) \in O(n^2)$$

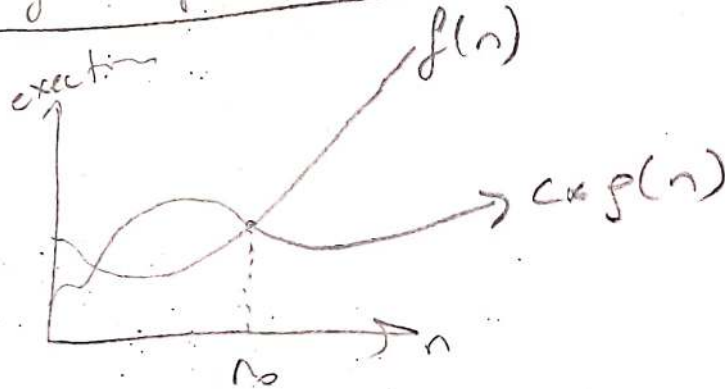
$$n^2 + n \in O(n^2) //$$

①

Algo

B. 10.21

Big-Omega Notation $\Rightarrow$ Asymptotic Lower Bound.



$$c > 0 \text{ ve } n_0 > 0$$

$$f(n) \geq c \cdot g(n)$$

$g(n) \Rightarrow$ asym lower bound

Ex 1 : $f(n) = 5n^2$ $g(n) = n^2$

$$5n^2 \in \Omega(n^2)$$

$$5n^2 \geq c \cdot n^2$$

$$n^2(5 - c) \geq 0$$

$$n_0 = 1, c = 1 \Rightarrow 4 \geq 0 \checkmark$$

$$n_0 = 2 \rightarrow 8 \geq 0 \checkmark$$

$$n_0 = 3 \rightarrow 12 \geq 0 \checkmark$$

$\rightarrow$ for all $n \geq n_0$
 $f(n) \in \Omega(g(n))$
 $5n^2 \in \Omega(n^2) //$

$5n^2 \in \Omega(n) \Rightarrow$ Evet dogru ifade

$$5n^2 \geq cn$$

$$5n^2 - cn \geq 0$$

$$n(5n - c) \geq 0$$

$$n_0 = 1, c = 1 \Rightarrow 4 \geq 0 \checkmark$$

$$n_0 = 2 \Rightarrow 18 \geq 0 \checkmark$$

$$n_0 = 3 \Rightarrow 42 \geq 0 \checkmark$$

①

Algo Anal

20.10

Best Case
Worst Case
Avg Case

$\sum_{i=1}^n 1 = n$	$\sum_{i=l}^u 1 = u-l+1$
$\sum_{i=1}^n i = \frac{(n+1) \cdot n}{2}$	$\sum_{i=1}^n i^2 = \frac{1}{3} n^3$

```
int findMax(int n, int A[])
```

```
{ int i, max;
```

```
max = A[0];
```

```
for (i=1; i<n; i++)
```

```
{ if (max < A[i])
  max = A[i];
```

```
}
```

```
return max;
```

```
}
```

Basic operation

$$C(n) = \sum_{i=1}^{n-1} 1 = n-1$$

number of comparison $O(n)$

Best = Worst = Avg = $n \Rightarrow \Theta(n), O(n)$

```
int sequentialSearch(int n, int A[], int x)
```

```
{ int i=0
  while ((i<n) && A[i] != x)
```

```
    i++;
```

```
  if (i<n)
```

```
    return i;
```

```
  else return -1;
```

```
}
```

$$\frac{P(n+1)}{2} + n(1-P)$$

Best case = 1 $\Rightarrow O(1)$
Worst case = $n \Rightarrow O(n)$

Avg case = successful search = $P \Rightarrow 0 \leq P \leq 1$

$$i^{\text{th}} \text{ position} = \frac{P}{n}$$

$$C_{\text{avg}}(n) = \left[1 \times \frac{P}{n} + 2 \times \frac{P}{n} + 3 \times \frac{P}{n} + \dots + n \times \frac{P}{n} \right] + n \cdot (1-P)$$

$$= \frac{P}{n} (1+2+\dots+n) + n \cdot (1-P)$$

$$P/n \cdot \frac{n \cdot (n+1)}{2} + n \cdot (1-P)$$

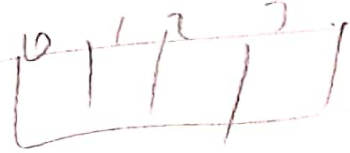
①

Algo Anal

27.10.21

Non-Recursive Algorithms

Insertion_Sort ayak



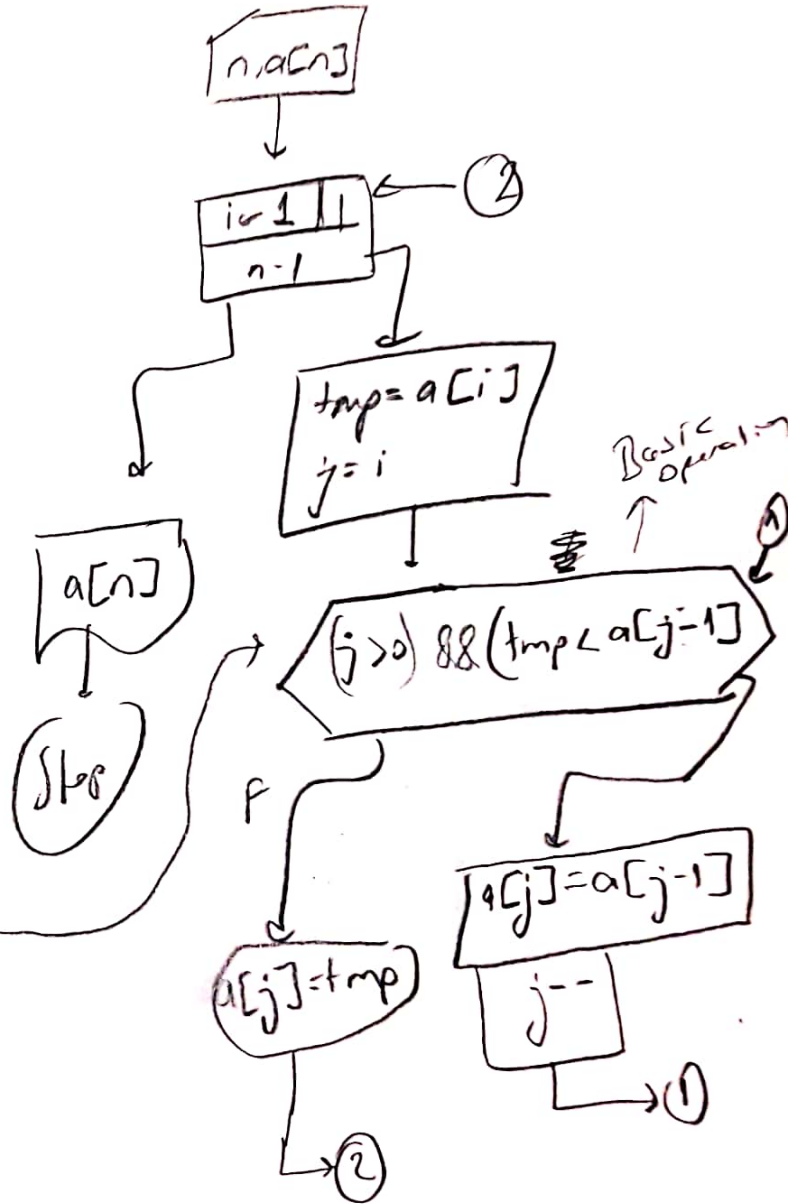
Worst case: trastinsirali oluncadir.

Basic op.
i here girer 2x i

$$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$$

$$= \frac{n^2}{2} \in O(n^2)$$

$$\in \Theta(n^2)$$

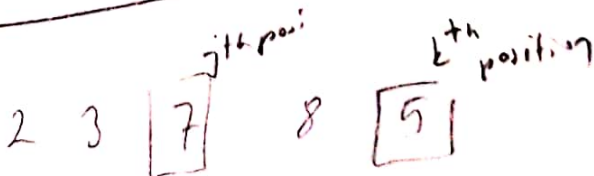


Best case

Normal sirali oluncadir.

$$\sum_{i=1}^{n-1} 1 = \textcircled{n-1} \in O(n)$$

avg case =



if final pos is j:
k-j+1 comparison.

on avg $\frac{1}{k} \cdot \sum_{j=1}^k (k-j+1) = \frac{1}{k} \left[k^2 - \frac{k(k+1)}{2} + k \right] = k - \frac{k+1}{2} + 1 = \frac{k+1}{2}$

after $\sum_{k=1}^{n-1} \frac{k+1}{2} \rightarrow \text{stat1} \rightarrow \text{bruc} \approx \frac{n^2}{4} \Rightarrow \in O(n^2)$

(2)

Algo Analysis

27.10

Recursive Algorithms Calculation of Complexity

$$\text{faktoriyel hesobi} = n! = n \cdot (n-1) \cdot \dots \cdot (1)$$

$$n! = n \cdot (n-1)!$$

$$0! = 1$$

$$\text{fact}(n) = \begin{cases} 1, & n=0 \\ n * \text{fact}(n-1), & \text{otherwise} \\ & (n \geq 1) \end{cases} \rightarrow \text{Recurrence relation}$$

$$M(n) = \underbrace{M(n-1)}_{\text{fact}(n-1)} + 1 \rightarrow n * \text{fact}(n-1)$$

1) Backward Substitution

$$M(n) = M(n-1) + 1 = M(n-2) + 2 \Rightarrow M(n) = M(n-3) + 3$$

$$M(n-1) = M(n-2) + 1$$

$$M(n-2) = M(n-3) + 1$$

$$\vdots$$

$$M(n) = M(n-i) + i$$

$$M(0) = 1$$

$$M_{\text{fact}}(n) \in \Theta(n)$$

$$n-i=0$$

$$i=n$$

$$M(n) = M(0) + n$$

$$M(n) = 1 + n$$

$$M(n) \in O(n)$$

3

Algo Anal

27.10

② forward Substitution

$$T(n) = \begin{cases} 1, & \text{if } n=1 \\ 2T(n-1)+1, & \text{otherwise} \end{cases}$$

n	$T(n)$
1	1
2	$2T(2-1)+1 = 3$
3	$2.T(3-1)+1 = 7$
4	$2.T(4-1)+1 = 15$
i	$\rightarrow 2^i - 1$
n	$\rightarrow 2^n + 1$

$$T(n) = 2^n + 1 \in O(2^n)$$

Master Theorem Asgirdaki sentler saglayan recurrence relation lar aytayca yor ar mesela forward/backward sub ile aytayca bu yordamlar olaklar.

$a \geq 1, b \geq 1, c > 0$; constant
 $T(1) = c$

$$T(n) = a * T\left(\frac{n}{b}\right) + f(n)$$

of subproblems size of each subproblem relative to n cost of combining

$$T(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{1/b^d}) & \text{if } a > b^d \end{cases}$$

ör $T(n) = T\left(\frac{3n}{4}\right) + n$
 $a=1, b=4/3, f(n)=n^1 \rightarrow \underline{d=1}$
 $b^d = 4/3 \rightarrow \underline{a < b^d}$
 $T(n) \in O(n^1)$

ör $T(n) = T\left(\frac{2n}{3}\right) + 1$ $f(n)=1 \Rightarrow n^0$
 $a=1, b=3/2, b^d=1, \underline{a=b^d}$ $\underline{d=0}$
 $T(n) \in O(\log n)$

4.

Algo Analizi

27.10

~~Q1~~ Decimal to Binary Conversion sonrası kaç dıgıttır? Gerçekli recursive kodu yaz. Recurrence relation ır. sonra Complexity nedir?

①

Algo Anal

3.11.21

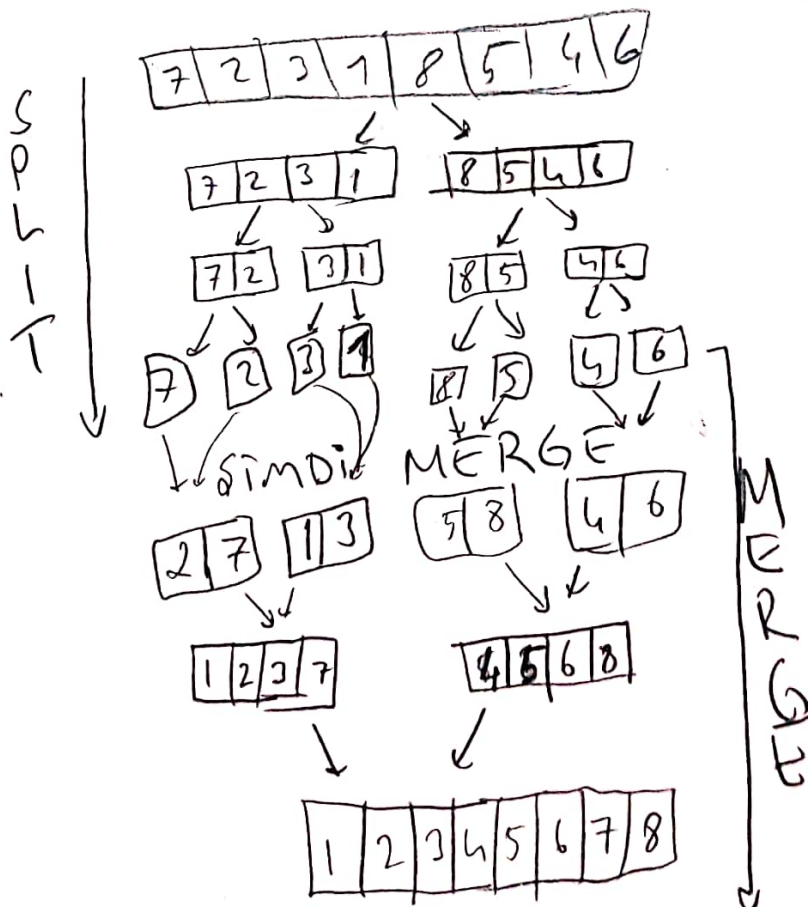
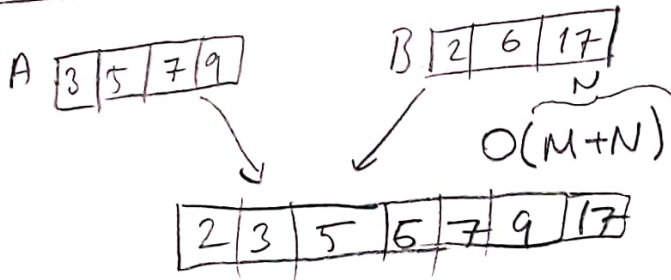
$O(n^2)$

Divide And Conq

Quicksort - Mergesort $\Rightarrow$ Popular ones

Merge Sort

merge oper'n $\Rightarrow$ worst time oper'n

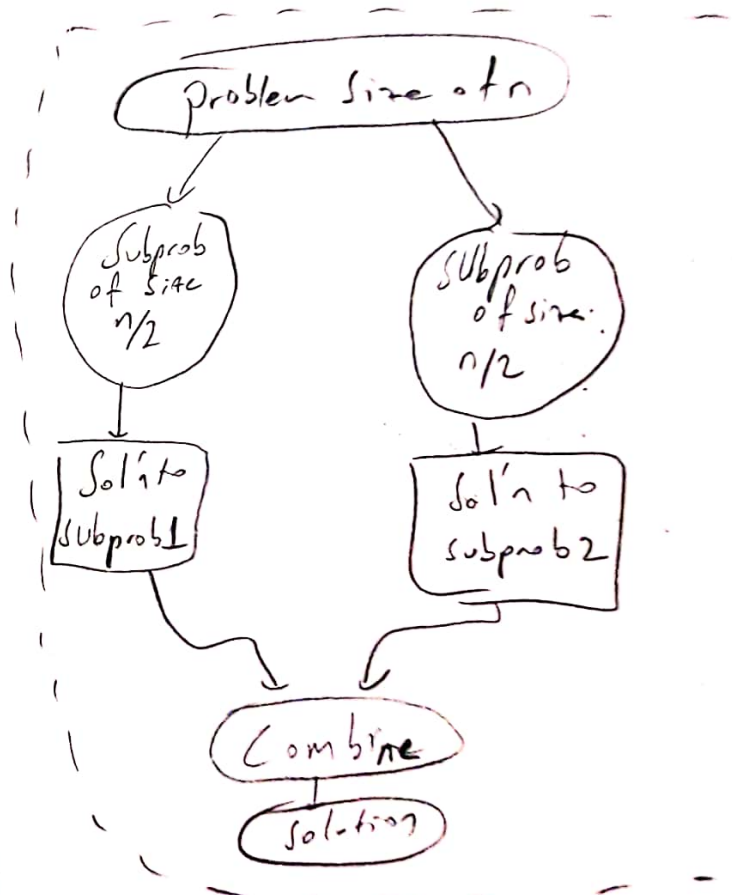


(*) Selection Sort $O(n^2)$

1) Divide: divide the prob. into smaller problems

2) Conquer: solve these subproblems

3) Combine: combine results together



(2)

Algo Anal

3.11.21

```

void mergeSort(int A[], int l, int r)
{
    if (l < r)
    {
        q = (l+r)/2;
        mergeSort(A, l, q);
        mergeSort(A, q+1, r);
        Merge(A, l, q, r);
    }
}

```

```

void merge(int A[], int l, int m, int r)
{
    int *B;
    int i, j, k;
    int n = r - l + 1; // num of elements
    B = (int*) malloc(n * sizeof(int));
    i = l; j = m+1; k = 0;
    while (i <= m && j <= r)
    {
        if (A[i] < A[j])
            B[k++] = A[i++];
        else
            B[k++] = A[j++];
    }
    while (i <= m)
        B[k++] = A[i++];
    while (j <= r)
        B[k++] = A[j++];
    Copy B to A;
    Free B;
}

```

Recurrence Relation of MS

$$T(n) = \begin{cases} O(1) & \text{if } n=1 \\ 2 \times T(n/2) + n & \text{otherwise} \end{cases}$$

$$\begin{aligned}
 T(n) &= 2 \times T(n/2) + n \\
 &= 2 \times [2 \times T(n/4) + n/2] + n \\
 &= 4 \times T(n/4) + 2n \\
 &= 4 \times [2 \times T(n/8) + n/4] + 2n \\
 &= 8 \times T(n/8) + 3n \\
 &= 2^i \times T(n/2^i) + i \times n
 \end{aligned}$$

$$\frac{n}{2^i} = 1 \Rightarrow 2^i = n \Rightarrow i = \log_2 n$$

$$\Rightarrow 2^{\log_2 n} \times T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n \times n$$

$$\Rightarrow n \times T(1) + \log_2 n \times n$$

$$T(n) = n \times 1 + \log_2 n \times n$$

$$\in (O(n \times \log_2 n))$$

$$\in \Theta(n \times \log_2 n)$$

Quick sort ya gadi.

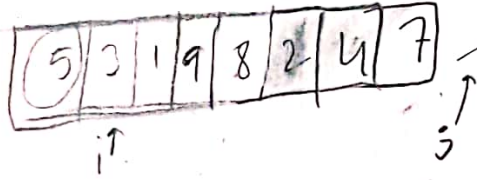
EO Lecture.

①

Algo Anal

10.11.21

Quick Sort



int partition(int a[], int l, int r) $\rightarrow$ initial: $l=0, r=n-1$

{ int pivot, i, j;

 pivot = a[l];

 i = l, j = r+1;

 do

 { do

 { ++i;

 } while((i < j) && (a[i] <= pivot))

 } do

 { --j;

 } while(a[j] > pivot)

 xchp(&a[i], &a[j]);

 } while(i < j)

 xchp(&a[i], &a[j]); // undo last exchp. because hata yapiyor sonradan

 xchp(&a[j], &a[l]); // pivotu dogru yere koy.

 return j;

}

Stabil Sort
YA Z

②

Algo Analysis

10.11.21

```
void QuickSort ( int a[], int l, int r) //initial l=0, r=n-1
{
    int pivotaddr;
    if ( l < r )
    {
        pivot = partition ( a, l, r );
        QuickSort ( a, l, pivotaddr - 1 );
        QuickSort ( a, pivotaddr + 1, r );
    }
}
```

④ Complexity analysis:

① Best Case: Pivot is in the middle at each partitioning.

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2 * T(n/2) + n & \text{otherwise} \end{cases}$$

partition is karmunkhigi

Best Case $T(n) = \Theta(n \log_2 n)$

② Worst Case: when the array is SORTED!

1 3 5 8 10

$$T(n) = T(n-1) + n \leftarrow \text{partition}$$

$\Rightarrow \Theta(n^2)$

③ Avg Case:

$$T(n) = 1.39 * N \log_2 n$$

$\in O(n \log_2 n)$

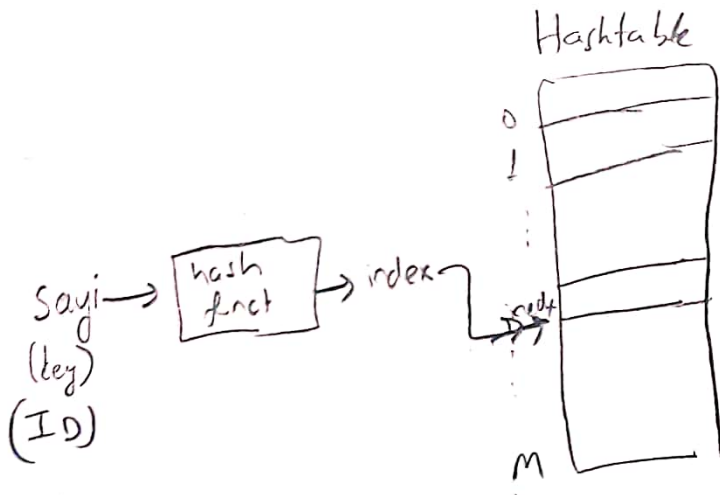
(3)

Algo Anal

10.11.21

Hashing $\rightarrow O(1)$ // karmaşıklık 'n'e bağıl **DEĞİL!!!**
 // Arama, Ekleme, Silme işlemleri

Direct access sağlanıyor kütüphane.



	Search	Insert	Del
	W	W	W
	W	W	W
	W	W	W
Sequential Search	N	1	N
Binary Search	$\log_2 N$	$\log_2 N$	$\log_2 N$
BST	N	$\log_2 N$	$\log_2 N$

Hash functions

① Division Method
 func $\Rightarrow$ sayı mod m ;

ör $x=25$ table size = 7

$x=25$
 $\text{index} = 25 \% 7 = 4$

② Multiplication Method.

$\text{hash}(x) = \frac{m * [(x * A) \bmod 1]}{\text{table size}}$
 $\frac{5-1}{2} = 0,18033$

ör $x=123456$, $m=10^5$

$\text{hash}(123456) = 10^5 * \left[\frac{123456 * 0,18033}{\text{mod } 1} \right]$

$= 10^5 * 0,0041151$

$= 41$

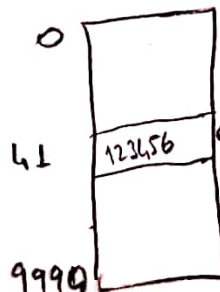


Table Size ne olmalı?

$m \gg N$ olmalı

$m \approx 2 * N$

$2 * N$ civarı 1 asal sayı

olmalı **M**

④ hash func's continuation Algo Anal

10.11.21

③ horner method (31)

Elif $\rightarrow$ key veya ID olsun.

length("Elif") $\rightarrow$ L olsun.

$$\text{hash}(\text{"Elif"}) = 31^{L-1} * \text{num}('E') + 31^{L-2} * \text{num}('l') + \dots + 31^0 * \text{num}('f')$$

31 veya asal başka sayılar.

$$\text{num}('E') = 'E' - 'A' + 1$$

$\uparrow$
'a' harfini
yutması
dige

key int olabilir sorun yok ✓
string olabilir ✓
unique olması lazım
float olabilir tersiye çevir

key = 35.84 $\rightarrow$ round(x * 10000) % 0000

input size
 $N < M \rightarrow$ table size
 $M \approx 2 * N$

①

Algo Anal

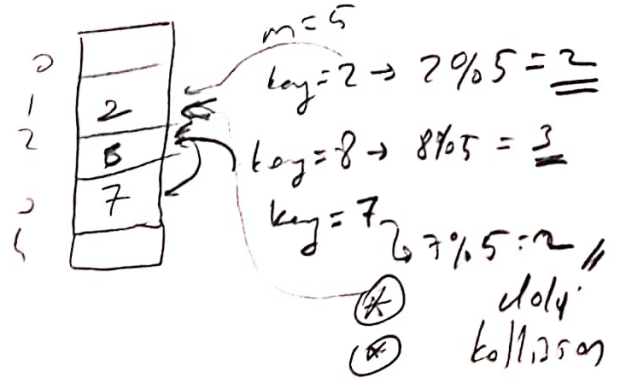
17.11.21

Collision Problem

① Open Addressing

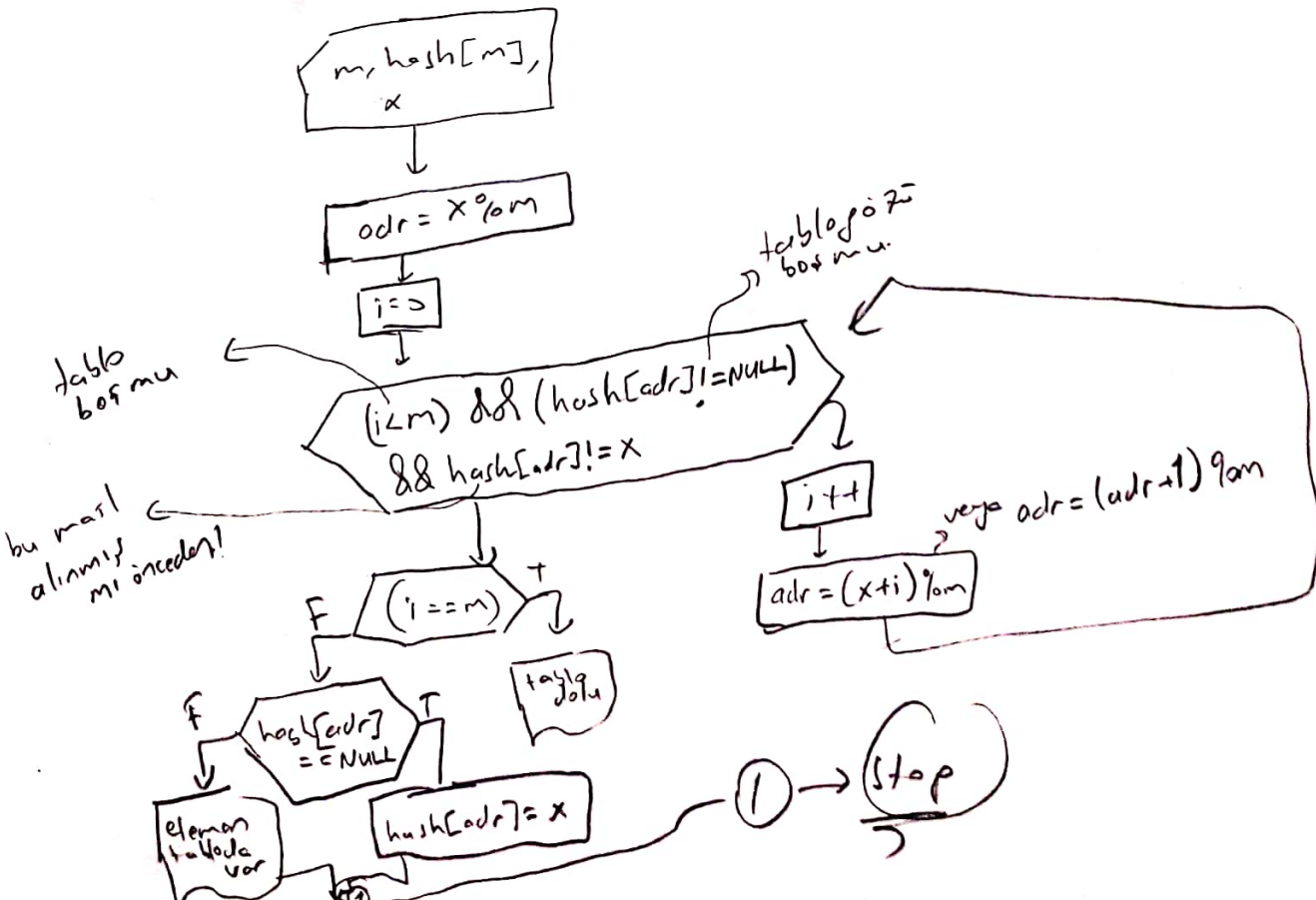
1-1 linear probing → each check is a control

$$\text{hash}(key, i) = \text{hash}'(key) + i \% m$$



$$\text{hash}'(key) = key \% m$$

② $Gr = m$ uzunluğunda tablo. Gelen bilgiler division metoduyla ve calışır durumda linear probing yöntemiyle yerleştiriliyor. Kullanıcılar nasıl adr. verir? program ile bir adres veya nesne veren, yoksa bu nasıl adresini tabloya ekleyen işlemi gerçekleştirir.



(2)

Algo Anal

17.11.21

1/1 * Delete Operasyonu hash tablosuna struct olarak implemente ederse işi işi kolaylaştır.

Table	Flag
	Deleted
	Empty
	Occupied

* Linear probe'de adın büyüklüğüne göre chunk chunk yapılmaz. Sebabi. Gere Double hashing

1-2 * Double hashing

$$\text{hash}(\text{key}, i) = [\text{hash1}(\text{key}) + i * \text{hash2}(\text{key})] \% m$$

i^{th} step

$m = \text{table size}$
 $m = 13$

$$\text{hash1}(\text{key}) = \text{key} \% 13$$

$$\text{hash2}(\text{key}) = \text{key} \% 11 \rightarrow \text{hash2 bite step size verir.}$$

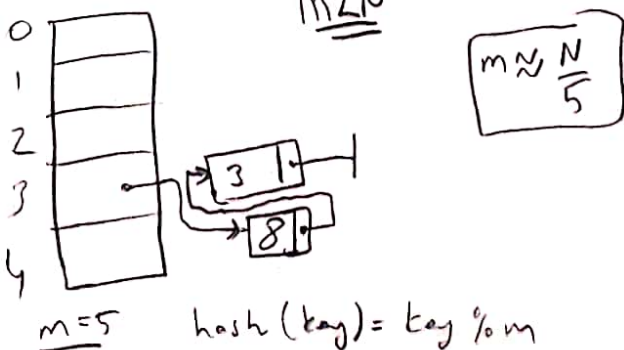
Load Factor

$\alpha = \text{tablo doluluk oranı}$

$$\alpha = \frac{N}{M} \quad \left| \quad M \approx 2N \right| \leftarrow \text{open addressing}$$

$2N$ 'e en yakın asal sayı olsun m

2/2 * Separate Chaining (Closed Hashing) * Çıkarsa olursa linked list yapıyoruz



$$\text{hash}(3) = 3 \% 5 = 3$$

$$\text{hash}(8) = 8 \% 5 = 3$$

Dolus En bas elemanı kopar. En bası yarıdır.

(3)

Algo Anal

17.11.21

```
typedef struct Node
{
    int key;
    char name[100];
    struct Node *next;
} n;
```

```
typedef struct hash
{
    struct Node *head;
    int count;
} h;
```

```
h *hashtable = NULL;
```

```
int main()
```

```
    int key;
    char name;
```

```
    hashtable = (h*) malloc (↑ table size - M * sizeof(h));
```

```
    !
    insert to Table (key, name)
```

```
    !
```

```
void inserttoTable (int key, char name[])
{
    int hashindex; // global var.
    hashindex = key % M; // allocation ops.
    n *newNode = createNode (key, name);
    if (!hashtable[hashindex].head)
    {
        hashtable[hashindex].head = newNode;
        hashtable[hashindex].count = 1;
    }
    else
    {
        newNode->next = hashtable[hashindex].head;
        hashtable[hashindex].head = newNode;
        hashtable[hashindex].count++;
    }
}
```

```
n *createNode (int key, char [])
{
    n *newnode;
    newnode = (n*) malloc (...);
    // kontrol
    newnode->key = key;
    strcpy (newnode->name, name);
    newnode->next = NULL;
    return newnode;
}
```

(16)

Algo Anal

17.11.21

③ Universal Hashing

$h_1()$

$h_k()$

$$h_{a,b}(key) = [(a \cdot key + b) \% p] \% m$$

$m = \text{table size}$

$a, b, p = \text{variable}.$

①

Algo Anal

1.12.21

Open Addressing

asıl sayı...

$$M \geq 2 \times N$$

element



Load factor: $\alpha = \frac{N}{M}$

$N \rightarrow$ # of elements
 $M \rightarrow$ table size

$\alpha = 1$ ne table is full

$\alpha = 0$ ne table is empty

$P(\text{first location is occupied}) = \alpha$: load factor

$P(\text{first location is empty}) = 1 - \alpha$

$P(\text{second location is empty}) = \alpha \times (1 - \alpha)$

$\downarrow$ 1st is full

$P(\text{3rd loca. is empty}) = \alpha \times \alpha \times (1 - \alpha)$

$P(k^{\text{th}} \text{ is empty}) = \alpha^{k-1} \times (1 - \alpha)$

$E(\# \text{ steps}) = \sum_{k=1}^m k \times \alpha^{k-1} \times (1 - \alpha) \approx \frac{1}{2} \times \left(1 + \frac{1}{1 - \alpha}\right) \rightarrow O\left(\frac{1 + \alpha}{1 - \alpha}\right)$

$\downarrow$
for empty cell

$\frac{N}{M}$
 $= O(1)$

Seperate Chaining



$$M = \frac{N}{5}$$

$$\frac{N}{10} = M$$

olması gerekiyor

Table size Hardcode asal sayı olmalı.

②

Algo Anad

1.12.21

Applications of Hashing

• Spell checker	key word	in list dictionary
• web browser	URL	visited pages.
• spam filter	IP	spam addresses.
• fraud detection	card number	stolen cards list.

★ Arama yapılması için uygularda kullanılır. Çok hızlı.

	<u>Worst Case</u>			<u>Avg Case</u>		
	S	I	D	S	I	D
Sorted Array	$\log N$	N	N	$\log N$	$N/2$	$N/2$
Unsorted Array	N	N	N	$N/2$	N	$N/2$
Linear Probing	N	N	N	1	1	1
Separate Chaining	N	1 ↓ Baki elemler	N	1	1	1

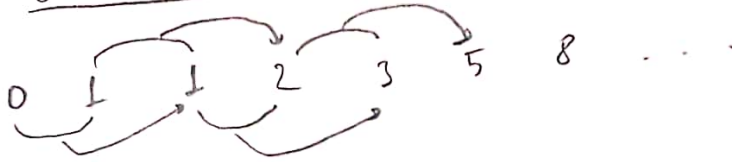
$O(1)$ yani

(3)

Algo Anal

1.12.21

⊛ QuickSortu iterative yap.

Fibonacci Numbers

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

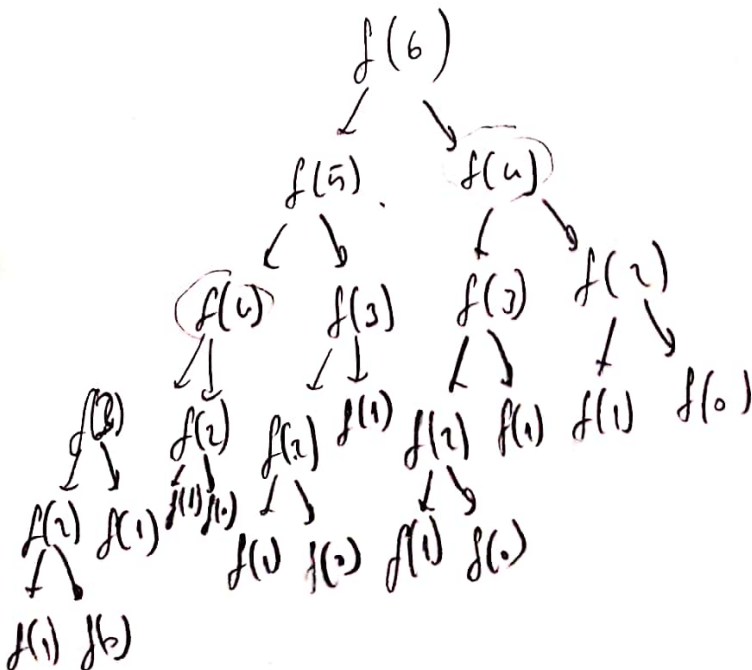
$$\text{fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{fib}(n-1) + \text{fib}(n-2), & \text{if } n \geq 2 \end{cases}$$

```

int fib(n)
{
    if(n==0)
        return 0;
    if(n==1)
        return 1;
    return fib(n-1) + fib(n-2);
}

```

⊛ Aynı şeyi tekrar tekrar hesaplamak zorunda kalıyoruz. Bu yanlış. Hesaplanan sonuçları saklarsak aynı alt problem-i tekrar tekrar çözüm zorunda kalmayız.



Birçok kere $\text{fib}(3)$, $\text{fib}(2)$, $\text{fib}(1)$, $\text{fib}(0)$ hesaplandı. Yanlış.

1 kere hesap sonucu sakla. Dinamik

4

Algo Anal

1.12.21

DYNAMIC PROGRAMMING → planning dernek

Her subproblem dependent ise kullanılır.

① Computing binomial coefficients.

$$(x+y)^2 = x^2 + 2xy + y^2 \rightarrow 1 \quad 2 \quad 1$$

$$(x+y)^3 = \dots \rightarrow 1 \quad 3 \quad 3 \quad 1$$

$$(x+y)^4 = \dots \rightarrow 1 \quad 4 \quad 6 \quad 4 \quad 1$$

$$(x+y)^n = C(n,0)x^ny^0 + C(n,1)x^{n-1}y^1 + C(n,2)x^{n-2}y^2 + \dots + C(n,k)x^ky^{n-k} + \dots + C(n,n)x^0y^n$$

$$C(n,i) = \frac{n!}{i! \cdot (n-i)!}$$

$$C(n,k) = C(n-1, k-1) + C(n-1, k)$$

recurrence relation =

$$C(n,k) = \begin{cases} 1 & ; \text{ if } n=k \text{ or } k=0 \\ C(n-1, k-1) + C(n-1, k) & , \text{ if } \underline{n, k} > 0 \end{cases}$$

Birbirine bağlı subproblem'in çözümü:
 farklı / metriste saklanır.

5

Algo Anal

1.12.21

exhaustive search open

coin = [1, 2, 5]

KNAPSACK PROBLEM:

knapsack = W

$1 \leq i \leq n$ — weights: $w_1, w_2, \dots, w_n$
 Values: $v_1, v_2, \dots, v_n$

maximize $\sum v_i$ subject to W
 $\sum w_i \leq W$

Item	value	Weight
1	2	12
2	1	10
3	3	20
4	4	15

$W = 50$

		weights					
		0	1	2	3	4	5
Items	0	0	0	0	0	0	0
	1	0					
	2	0					
	3	0					
	4	0					

$P(i, w)$ = i^{th} row
 and w^{th} column
 profit!!!

Parabola Problem:

Elde (1, 2, 5) + 11 kupa

11 tane 1 ya on at sayida para ile oluster.

cevap → 2 tane 5 1 tane 1
 veya

11 tane 1 + 1
 veya

9 tane 1 ve 2 tane 2
 veya

5 tane 2 ve 1 tane 1

⇒

0 1 2 3 4 5

Recurrence relation:

$$C(p) = \begin{cases} 0 & \text{if } p = 0 \\ \min \{1 + C[p - \text{coin}[i]]\} & \text{if } p > 0 \end{cases}$$

$i \in \text{coin}[i] \leq p$

$$C(9) = \min \begin{cases} C(9-1) + 1 \\ C(9-2) + 1 \\ C(9-5) + 1 \end{cases}$$

$$P(i, w) = P(i-1, w-w_i) + v_i$$

(6)

Algo Anal

1.12.21

Case 1: thief can take item i

$$P[i, w] : P[i-1, w-w_i] + v_i \rightarrow$$

Case 2: thief can not take item i

$$P[i, w] = P[i-1, w]$$

recurrence relation

$$P(i, w) = \begin{cases} P[0, j] = 0 & \text{if } i = 0 \\ P[i, 0] = 0 & \text{if } j = 0 \\ P(i-1, w), & \text{if } w_i > w \text{ (not taken)} \\ \max \left\{ \begin{array}{l} v_i + P(i-1, w-w_i) \quad \text{taken} \\ P(i-1, w) \quad \text{Not taken} \end{array} \right\} \end{cases}$$

①

Algo Anal

8.12.21

ELMA → ELAMA
 ↳ ELA
 ↳ ALMA



EDIT DISTANCE

EL - MA
 EL A m A
 ↓
 INSERT

EL MA
 EL B -
 ↓
 change

- ① insert
- ② delete
- ③ change

operationların
 birisi olma olasılığı

$$\text{dist}(\text{elama}, \text{elma}) = 1$$

$$\text{dist}(\text{elma}, \text{elb}) = 2$$

$$\text{dist}(\text{elma}, \text{ulma}) = 1$$

alignment in Computational biology

- A G G C T A T C
 T A G - C T - T G

$$\text{dist}(\text{tagcttg}, \text{aggtatc}) = 4$$

↳ belirsiz
 ↳ belirsiz
 $\times$ belirsiz
 $\rightarrow$ EDC[i, j]

x'te i harfine
 y'de j harfine
 gelene kadar

$$\text{minimum distance} = \text{ed}[i, j]$$

① (dictionarydeki kelimeleri karşılaştırmak için)

2

Algo No 1

8.12.21

Levenshtein Algorithm

	∅	A	R	T	S
∅	0	1	2	3	4
M	1				
A	2				
T	3				
H	4				
S	5				

→ $x = \text{null}$ ve $y = \text{"arts"}$ den

$x = \text{"maths"}$
 $y = \text{null}$ den

recurrence relations

$$ed[i][j] = \min \begin{cases} ed[i-1][j] + \text{cost}, \\ ed[i][j-1] + \text{cost}, \\ ed[i-1][j-1] + \text{costchange} \end{cases}$$

$$\text{costchange} = \begin{cases} 0, & \text{if } x[i] = y[j] \\ 1, & \text{if } x[i] \neq y[j] \end{cases}$$

initialization:

$$i=0 \quad ed[0][j] = j, \quad j=0 \dots n$$

$$j=0 \quad ed[i][0] = i, \quad i=0 \dots n$$

$$\text{time} = O(n \times m) = O(n^2)$$

$$\text{space} = O(n \times m) = O(n^2)$$

5

Also Analizi

8.12.21

Longest Common Subsequence

Don alphabet = {a, g, c, t}

x = atct gat

y = tgcata

LCS(x, y) = tcat
tgat

LCS(x[1..n], y[1..m])

{ int c[0..n][0..m]

for i = 0 to n do

{ c[i][0] = 0 ; label[i][0] = k; i

for j = 0 to m do

{ c[0][j] = 0 ; label[0][j] = k; j

for i = 1 to n do

{ for j = 1 to m do

{ if (x[i] == y[j])

{ c[i][j] = c[i-1][j-1] + 1;

label[i][j] = take xy = x[i] or y[j] is in the string

}

devamı artkale.

Subsequence =

kaynakam

kay

kama

ama

make

kaymak

aymak

ileriye doğru alıyoruz

her harf a b c d e f g h i j k l m n o p q r s t u v w x y z

forunda değiliz

ama aldıkları

m b n sırasını

karşılamak

forunda değiliz

②

Algo Anand

8.12.21

```
else if ( c[i-1][j] > c[i][j-1] )
```

```
{ c[i][j] = c[i-1][j]
```

```
  label[i][j] = skipx;
```

```
}
```

```
else
```

```
{
```

```
  c[i][j] = c[i][j-1]
```

```
  label[i][j] = skipy;
```

```
}
```

```
}
```

```
}
```

```
}
```

Greedy Technique
Opener
Method

Optimal Substructure Property: you can express solution to overall problem in terms of solving to subproblems. This is more general property of recursion. The essence of recursion is that you can decompose the overall problem into smaller pieces, whose solutions are then combined in some way to generate the solution to the overall problem.

for DP $\Rightarrow$ Overlapping Subproblems $Optimal = \text{Any subproblem; def value given;}$
 $\Rightarrow$ Optimal substructure $Optimal = \text{For problem recursive definition; subproblem; optimal substructure property;}$

A problem is said to have optimal Sub-structure property; if an optimal solution of given problem can be obtained by using optimal solutions of its subproblems.

get LCS(x, y, label)

```
{ i = n, j = m
```

```
while(i != 0 and j != 0)
```

```
  switch (label[i][j])
```

```
{ case addxy:
```

```
  add x[i] to front of
```

```
  string
```

```
  i-- j--
```

```
  break;
```

```
case skipx;
```

```
  i--;
```

```
  break;
```

```
case skipy;
```

```
  j--;
```

```
  break;
```

```
}
```

```
}
```

① Self

Algo Anal 10/12.21

DYNAMIC PROGRAMMING

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \rightarrow \text{matrixe Lyukers, ve sol ist copraz.}$$

Binomial Coefficients

$$\text{Pascals identity} = \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

$$\binom{n}{n} = 1 = \binom{n}{0}$$

j	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1					1					
6	1						1				

FAST METHOD

- First Solution
- Analyze the first solution
- Identify the Subproblems
- Turn the solution around (Turn around)

⊗ take a problem
break into sub problems.
then break subproblems into
subsubproblem until you
reach base condition.
by doing recursively.
Remember turning around
is looking at this recur-
sion viewing as a table.
then you take out the
recursion and even the
memoization entirely
by looking at table
this solution depends on
this and this sol'n.
this tells us how
we fill the table up.

Memoization = Top-Down

Tabulation = Bottom-up

0-1 Knapsack

V_{ij} = value of optimal knapsack involving elements $a_1, \dots, a_i$ with a weight capacity j

Inputs

Items = $a_1, \dots, a_n$
 Weights = $w_1, \dots, w_n$
 Values = $v_1, \dots, v_n$
 Capacity = W

Final problem Sol'n

$$V_{n,W}$$

take no elements

$i = 0, 1, 2, \dots, n$

take everything

$j = 0, 1, 2, \dots, W$

carry nothing

fill capacity

Base cases

$i = 0$ value = 0

$j = 0$ value = 0

final optimal value:

$$V_{n,W}$$

$V_{i,j}$ = you are now including or excluding item a_i (wt = w_i , value = v_i)
 • is taking a_i even feasible? (will it fit in our remaining capacity)
 our weight capacity is j right now. so $\Rightarrow j - w_i \geq 0$ feasible

we have choice
 take it don't take it

$j - w_i < 0 \rightarrow$ infeasible
 no choice you cannot take a_i

$$V_{i-1, j-w_i} + v_i$$

update $\rightarrow$

take a_i :
 weight capacity decreases.

$$j = j - w_i \text{ (remaining capacity)}$$

3

Self

Algo Anal.

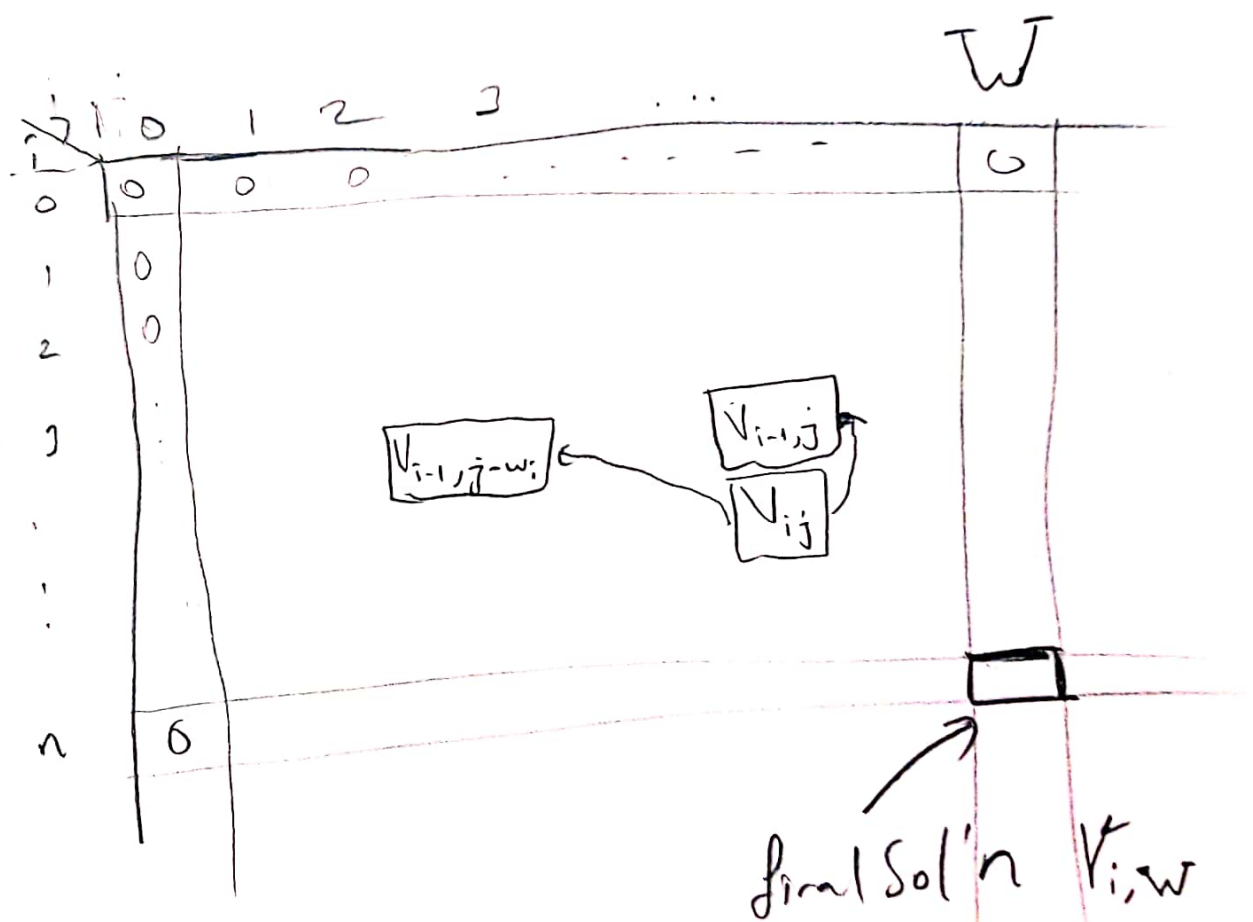
10.12.21

recurrence relation

$$V_{i,j} = \begin{cases} \max(V_{i-1,j}, V_{i-1,j-w_i} + v_i) & \text{if feasible}(j-w_i \geq 0) \\ V_{i-1,j} & \text{if } (j-w_i < 0) \end{cases}$$

choice

no choice



because $(j-w_i)$ is negative, then the first term is 0. So the recurrence relation is simplified to $V_{i,j} = V_{i-1,j}$. This is the base case of the recurrence relation.

(4)

Self

Algo Anal

10.12.21

Item	Weight	Value
a_1	5	10
a_2	2	5
a_3	4	8
a_4	2	7
a_5	3	7

 $W=7$ Pseudocode

```

for i = 0 .. n
  for j = 0 .. n
    if i = 0 or j = 0
       $V_{ij} \leftarrow 0$ 
    else if  $j - w_i < 0$ 
       $V_{ij} \leftarrow V_{i-1, j}$ 
    else if  $V_{i-1, j} > V_{i-1, j-w_i} + V_i$ 
       $V_{ij} \leftarrow V_{i-1, j}$ 
    else
       $V_{ij} \leftarrow V_{i-1, j-w_i} + V_i$ 
  }

```

$i \backslash j$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	10	10	10
2	0	0	5	5	5	10	10	15
3	0	0	5	5	8	10	13	15
4	0	0	7	7	12	12	15	17
5	0	0	7	7	12	14	15	19

Tableau/
Table.

Plus Ne uldigi-171 nası bulacagı 18?

Pseudocode orkank

(5)

Self

Algo Anal

10.12.21

Input = Tableau V of 0-1 Knapsack

Outputs: Optimal knapsack $S \subseteq A = \{a_1, \dots, a_n\}$

$S \leftarrow \emptyset$

$i \leftarrow n$ // start at the bottom

$j \leftarrow w$ // start at last column.

while $i \geq 0$ and $j > 0$ // stay in bounds of the table

{ while $i \geq 1$ and $V_{i,j} = V_{i-1,j}$
 { $i--$ }

$S \leftarrow S \cup \{a_i\}$

$j = j - w_i$

$i = i - 1$

}

output S

Search for partitioning of a set that 2 subsets are containing
same sum of numbers.
non-contiguous palindrome.

①

Algo Analizi

15.12.21

GRAPH TRAVERSAL

① Topological sort graph un yönlü ile yapılır.

② 2 şekilde gösterilir
→ adjacency matrix ^{hızlı erişim}
→ adjacency list (linked list) ^{yolları gösterir}

③ Lapunluk için de matrix kullanmak mantıklı

④ recommendation system

- Depth First Search $\Rightarrow$ put unvisited vertices on STACK

- Breadth First Search $\Rightarrow$ put unvisited vertices on QUEUE.

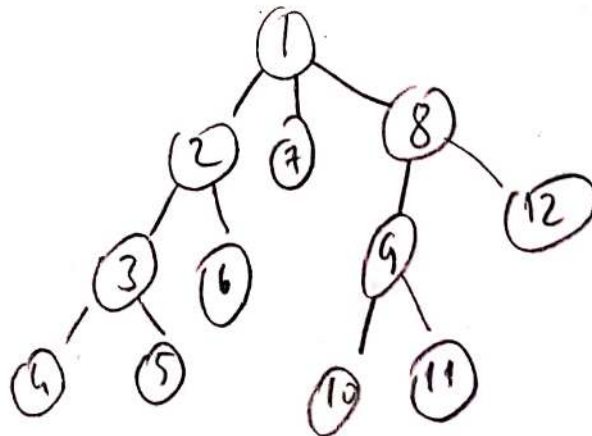
Stack $\Rightarrow$ Last In First Out.

DFS (to visit a vertex S)

mark S as visited

visit all unmarked vertices $\leftarrow$ adjacent to S.

recursive



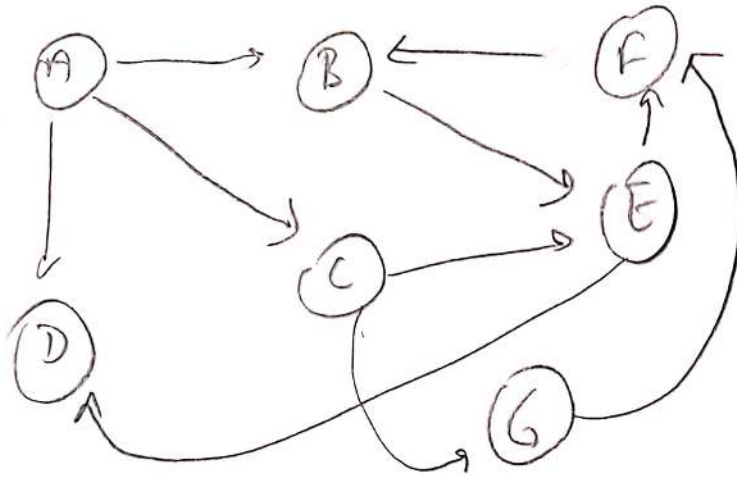
2

Algo Anal

15.12.21

④ Graf
üzünde
gezmek
sorusu.
Gibice

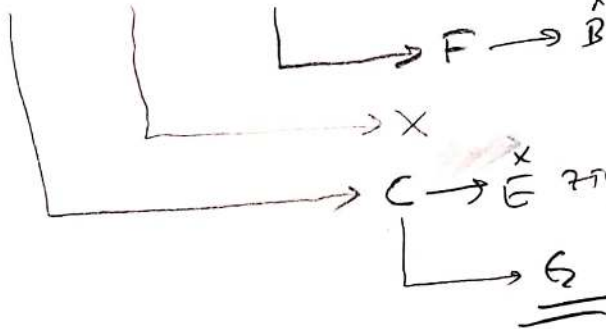
A'dan G'ye yol varmı?



ziyaret ettiklerini de
visited diye belirt
leveliye!

ABEDFC G

A → B → E → D



X → ziyaret edildi.

X ziyaret edildi.

yol var.

DFS (G, s)

{ mark s as visited

for all neighbours w of s in G

if w is not visited

- DFS (G, w)

}

→ Bir başlangıç noktasından ulaşılabilirlik için doğru-yanlış gözet.

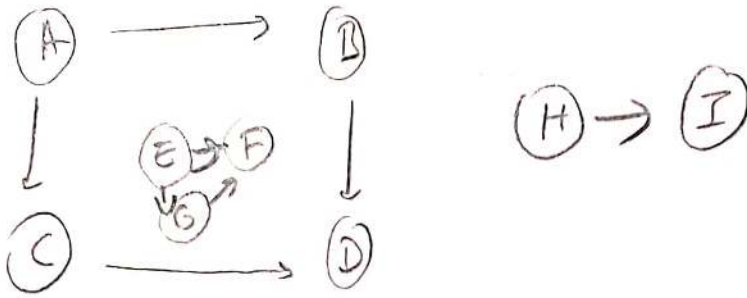
3

Algo Analysis

15.12.21

me

How connected component ver visited



vertex = 9

visited =

0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0
A	B	C	D	E	F	G	H	I

A B D C
comp-1

E F G
comp-2

H I
comp-3

Non recursive DFS using stack:

DFS (G, s)

```

{
  stack.push(s)
  mark s as visited
  while (stack is not empty)
  {
    v = stack.pop()
    for all neighbors w of v in Graph G
    {
      if w is not visited
      {
        stack.push(w)
        mark w as visited
      }
    }
  }
}

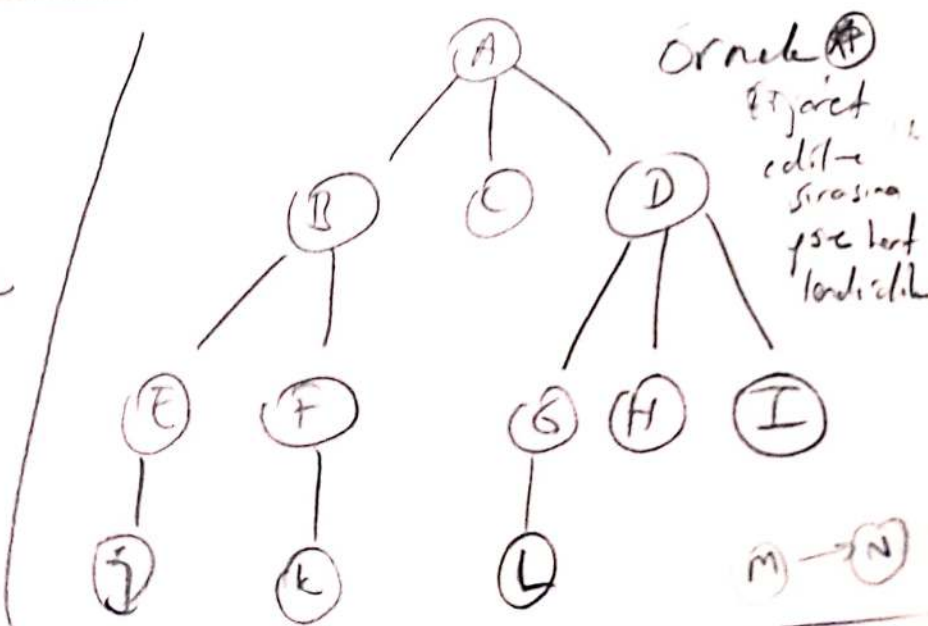
```

15.12.21

BFS (from source vertex s)

Repeat until queue B empty

- remove the least recently added vertex v
- add each of v 's unvisited neighbors to queue and mark them as visited



IFS (G, S)

$$\{ \text{Queue, enqueue}(s)$$

mark 5 as visited

while (Queue is not empty)

```

} v = Queue.dequeue();

```

for all neighbors w of v in graph G

If w is not visited

```

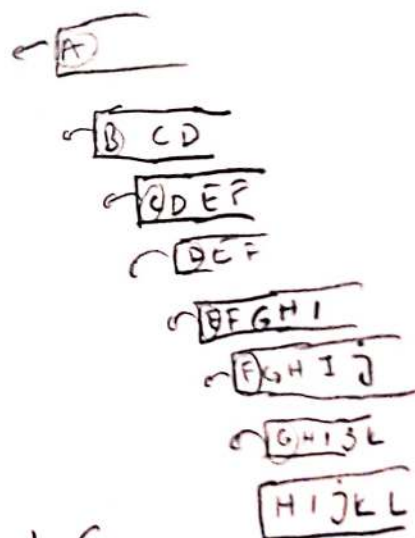
{ Queue.enqueue(w)
  mark w as visited
}

```

3

3

3



5

Algo Anal

15.12.21

L. S. J'larer s.ack prafitan A → K'ya en kusa yol?

A B C D E F G H I J K L

0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---

1 1 1

2 2 2 2 2

3 2

← A

← B C D

← D E F

← E F

← E F G H I

← F G H I J

G H I J K

pur.

① Self

Algo Anal

22.12.21

DYNAMIC PROGRAMMING w/ Tim Kearns

DP = defined by recurrences with overlapping subproblems

⊕ important idea is optimal substructure.

→ characterize the solution of problem in terms of the solution to smaller problems (subproblems).

→ after that write recurrence relation that captures the decomposition

→ record the solutions to subproblems. Use table!!

* Example coin-row problem (max weight indep. set in a path graph)

row of n coins w/ $(c_1, \dots, c_n) > 0$ integers.

Goal pick max amount of money BUT no two coins adjacent in the row can be picked up.

$C = (5, 1, 2, 10, 6, 3)$ $n=6$ number of coins.

⇒ Thought experiment: Suppose the problem contains

the 6 coins above, think like we know the solution for all the 6 coins; if we looked at the solution either the solution contains 6th coin or not contains 6th coin.

• if doesn't contain 6th coin, then the optimal solution must be the same with the optimal value of first 5 coins.

• if it does contain 6th coin then the optimal value of first 6 coins must be "the optimal value of first 4 coins + the value of 6th coin."

② self

Also Aral 22.12.21

Arka Sayfardali thought exp gives us recurrence relation.

Let $V(n)$ be the optimal value of picking from the first n coins then;

$$V(n) = \begin{cases} 0 & \text{if } n=0 \\ c_1 & \text{if } n=1 \\ \max(V(n-1), (V(n-2) + c_n)) \end{cases}$$

Reasoning About Optimal Substructure

● Thought experiments -

- Looking at the problem, what decision or options would have lead to a solution (in coin row, the sol'n either has or doesn't have the last coin.)
- if you know the best sol'n for $n-1$ and $n-2$ coins could you then determine what to do about n^{th} coin? YAY!

● Proof: Looking at the last coin, c_n in the row either it is or is not in the sol'n for $c_1, c_2, \dots, c_n$ let's call this S_n !

- Case 1 c_n is not in the S_n then the optimal sol'n must be the optimal sol'n for $n-1$ coins $\rightarrow S_{n-1}$
- Case 2 c_n is in the S_n then the optimal sol'n must contain the optimal sol'n for $n-2$ coins $\rightarrow S_{n-2} + c_n$

(3) Self

Algo Anal

22.12.21

Design & Implementation of D.P. Algo.

① important idea is optimal substructure of problem.

① characterize the sol'n to a problem in terms of the sol'n to smaller problems.

② Write down the recurrence relation that captures #1. Determine the table to hold the sol'n's to smaller versions of the problem (subprob's.)

③ Compute the optimal sol'n $\begin{cases} \rightarrow \text{using tabulation} \\ \rightarrow \text{using memoization} \end{cases}$

DFS(G, s)

```
{
  mark s as visited
  for all neighbours w of s in G
    if w is not visited
      DFS( $G, w$ )
}
```

DFS_nonrecursive(G, s)

```
{
  stack.push(s)
  mark s as visited
  while (stack is not empty)
  {
    v = stack.pop();
    for all neighbors w of v in G
    {
      if (w is not visited)
      {
        stack.push(w)
        mark w as visited
      }
    }
  }
}
```


④ Self

Algo Anant

22.12.21

BFS (G, s)

```
{
    Q.enqueue(s)
    mark s as visited
    while (Q is not empty)
    {
        v = Q.dequeue()
        for all neighbors w of v in G
        {
            if (w is not visited)
            {
                Q.enqueue(w)
                mark w as visited
            }
        }
    }
}
```

Source → dest karta kya adin?

```
int length-path (G, src, dest)
{
    adin[G → num of node] = {-1, -1, ...};
    qu.enqueue(src)
    mark src as visited
    adin[src] = 0;
    while (qu is not empty)
    {
        v = qu.dequeue();
        for all neighbors w of v
        {
            if (w is not visited)
            {
                qu.enqueue(w)
                mark w as visited
                adin[w] = adin[v] + 1
            }
        }
    }
    return adin[src];
}
```

connected component
sayi? ^{graph.}

```
int con-comp(G)
{
    int sayi = 0;
    int index;
    index = find_not_visited(G)
    while (index > -1)
    {
        DFS(G, index);
        sayi++;
        index = find_not_visited(G);
    }
    return sayi;
}
```

```
int find_not_visited(G)
int find_not_visited(G)
{
    int i = 0;
    int n = G → num of nodes;
    while (i < n && G → visited[i] == 1)
        i++;
    if (i < n)
        return i;
    else
        return -1;
}
```

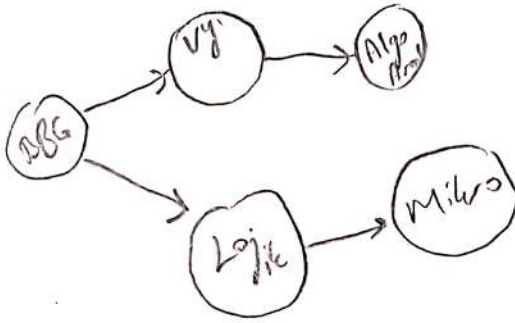

11

Algo Arac

29.12.21

DFS

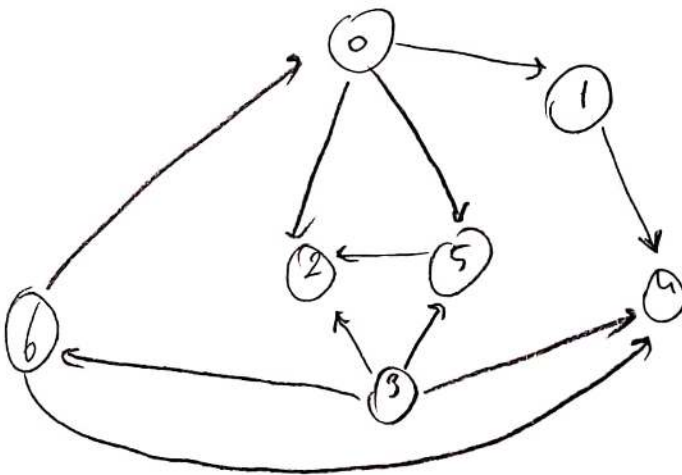
kullanarak Topologic Sort



işlerin hangi sıra ile yapılacağını gösterir.

⊗ Graf Directed
⊗ Acyclic olmalı.

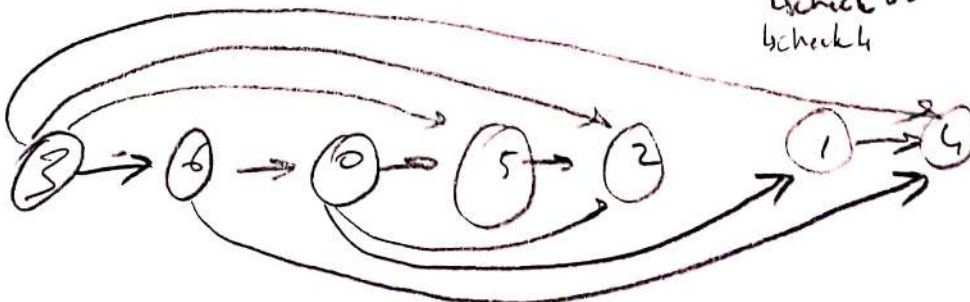
⊗ mutlaka source (başlangıç) noktası olmalı..



dfs(0) ⇒ done
↓
dfs(1) ⇒ done
↓
dfs(4) ⇒ done
↙
dfs(2) ⇒ done
↘
dfs(5) ⇒ 2 done idi 5 de done

3605 --- ⇒ Broken
3601 4 52) fazla
402-
olabilir

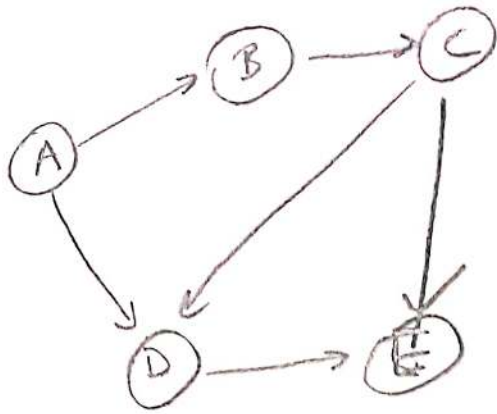
dfs(3) ⇒ done
↳ check 2 ✓
↳ check 5 ✓
↳ check 4 ✓
↘
dfs(6) ⇒ done ✓
↳ check 0 ✓
↳ check 4



(2)

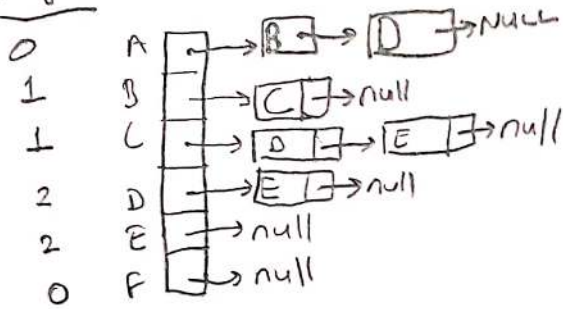
Algo Anal

29.12.21



(F)

In degree



A	B	C	D	E	F
0	1	1	2	2	0
	0	1	1	2	0
		0	1	2	0
			0	1	0
				0	0
					0

A'yı yaptık A'ya bağlı olanların indegree'sini 1 azalttık.

B'yi yaptık B'ne bağlı olanların indegree'sini 1 azalttık.

C'yi yap. C'ne bağlı olanların indegree'sini 1 azalttık.

D'yi yap. D'ne bağlı olanların indegree'sini 1 azalttık.

E'yi yap

F

① Store each vertex's in degree in an array $O(|E|)$

② while there are vertices remaining

- find a vertex 'w' in degree = zero

- reduce in degree of all vertices adjacent to "w" by 1 $\rightarrow O(|E|)$

- mark this vertex w's degree = -1

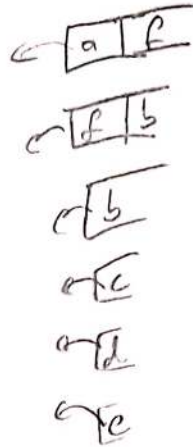
* in degree'lardan 0 olanları bulup, 0 olanı bulma işlemini tekrarlatabiliriz. Arka sırayla işleyebiliriz.

③

Algo Anal

29.12.21

in-degree	
0	a
1	b
1	c
2	d
2	e
0	f



⊛ Topological
Order/Sort
Sınacda
Gıka bılır

kyrkulu / Stackli
Algoru

- ① store each vertex's indegree $O(|E|)$
- ② initialize queue with all indegree=zero vertices $O(|V|)$
- ③ while there are vertices in queue
 - Dequeue and output $O(|V|)$
 - Reduce indegree of all vertices adjacent to it by 1 $O(|E|)$
 - Enqueue vertices with indegree=zero $O(|V|)$

⊛ $O(|V| + |E|)$

BACKTRACKING

DFS kullunılır baska sayı bala olabılır.

Brute force , Exhaustive Search $= (2^n)$

4

Algo Anal

29.12.21

N-Queen Problem

① Brute Force $\Rightarrow \binom{64}{8} = \frac{64!}{(64-8)! \cdot 8!} = 4,5 \text{ milyar possibilities}$

② One Per Row $\Rightarrow n^n = 8^8 = 16,7 \text{ milyon}$

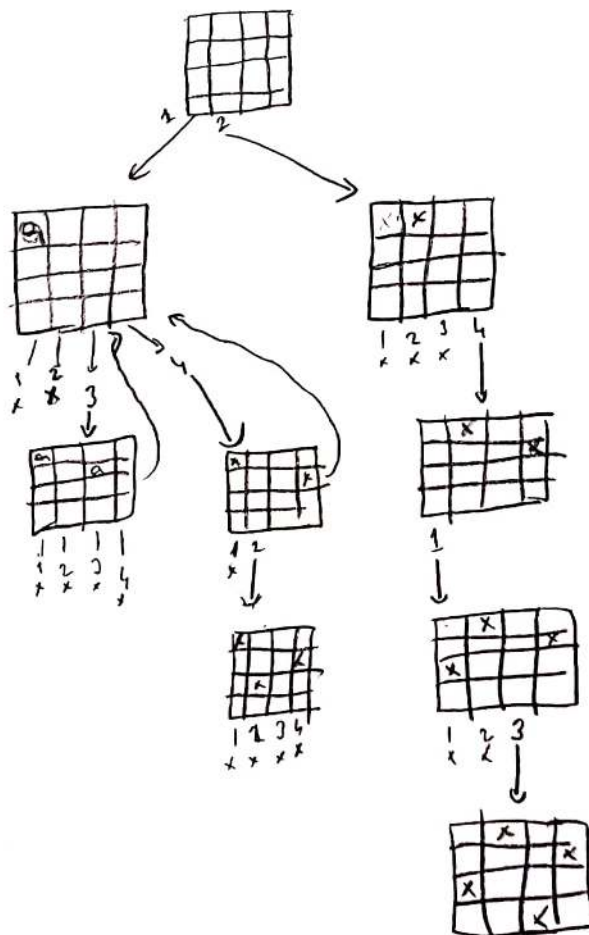
③ One Per Row and col $\Rightarrow n! = 8! = 40320$

④ Backtracking $\Rightarrow 2057 \text{ deneme}$

State Space Tree

4'lü tıng

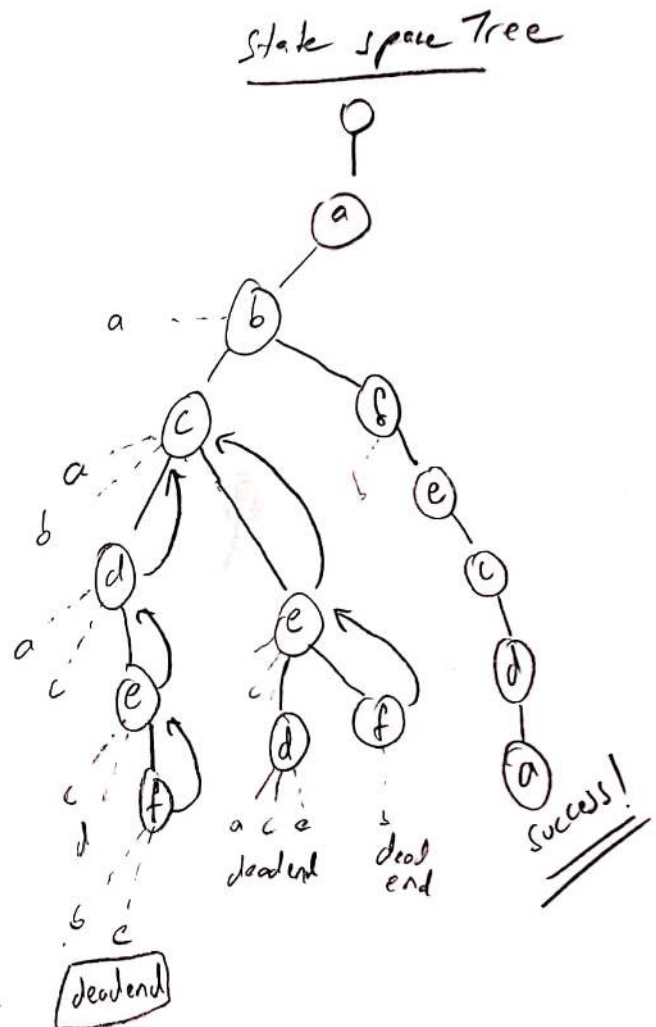
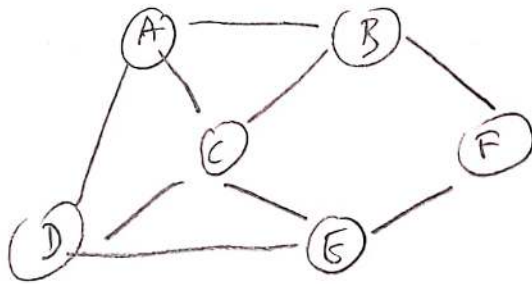
başlatıp...



Success!

Hamiltonian Circuit Problem

Bir kadın çıkıp geri gelsin. A — L getirdi — nede'ın 1'kine
uyrayayın.



①

Algo Anal

5.1.22

Branch & Bound

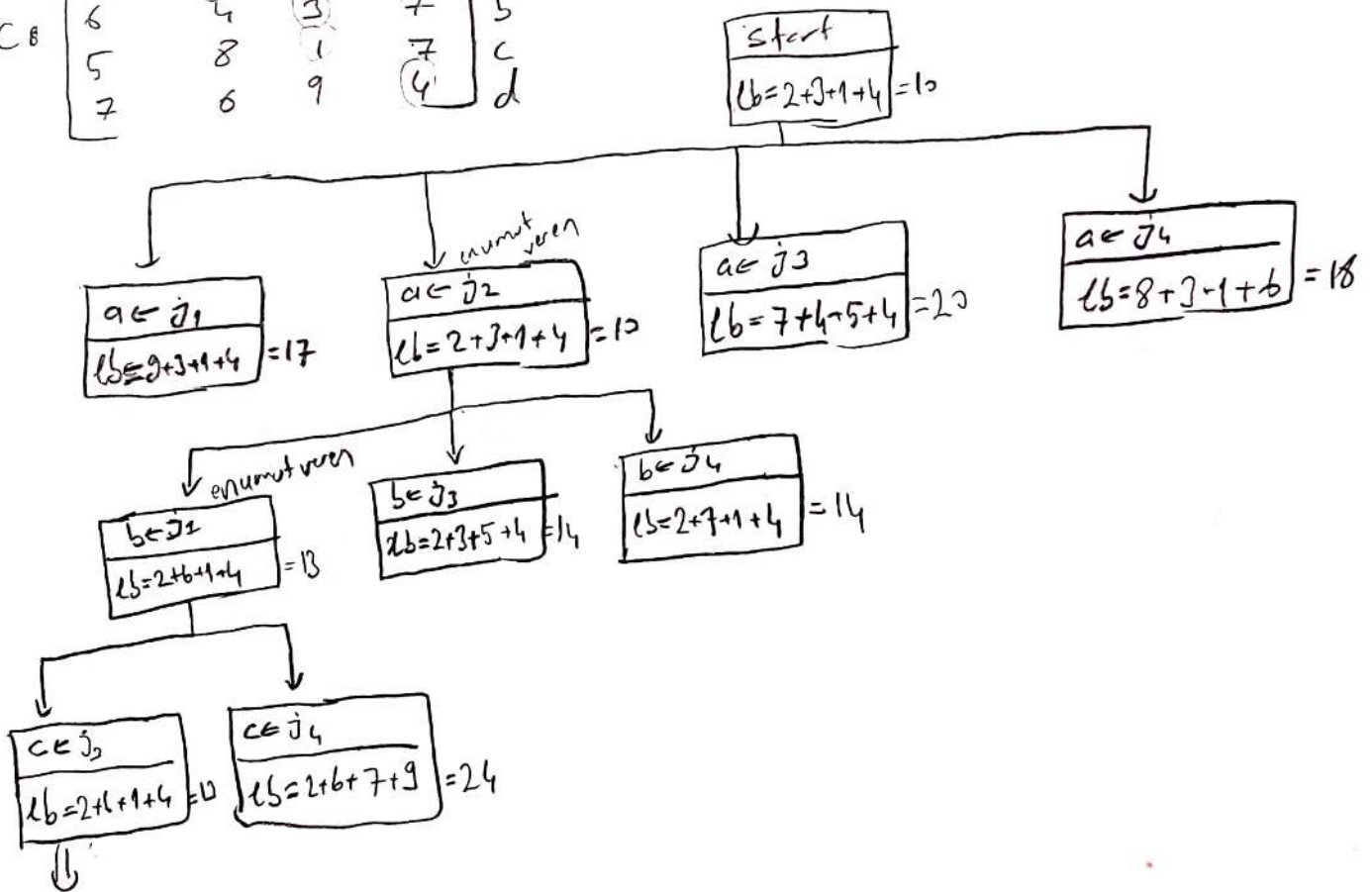
Backtracking uses
DFS

• uses Breadth First Search

① Job scheduling = lower bound BFS is used in Branch and Bound

	J_1	J_2	J_3	J_4	
C_1	9	2	7	8	a
	6	4	3	7	b
	5	8	1	7	c
	7	6	9	4	d

State space tree telefonda



daha
umut
vadeden yok!!!

(2)

Algo Aral 5.1.22

2-Knapsack : Upper bound.

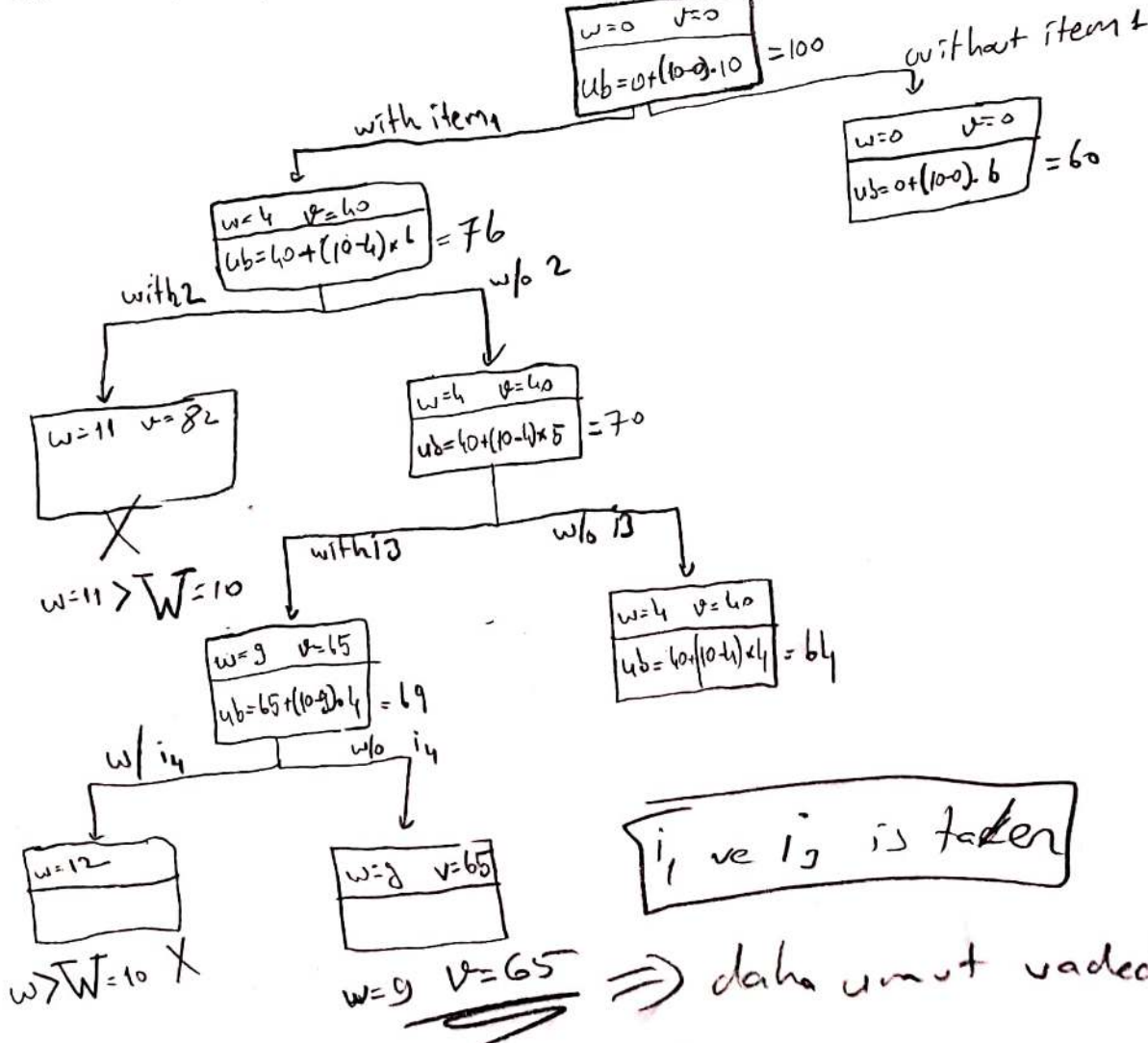
$W=10$ kg

→ Bunu göre sortla

item	weight	value	value/weight
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

$$Ub = v + (W-w)(v_{i-1}/w_{i-1})$$

State Space Tree → Stelde



For

i_1 ve i_2 is taken

① Self Algo Anal 8.1.22

FOR FINAL

GFG

1-) intro

Greedy Algo = an algorithmic paradigm that follows the problem-solving approach of making the locally optimal choice at each stage with the hope of finding a global optimum.

⊛ making locally optimal greedy choice DON'T always gives us a globally optimal value.

When to Use? Greedy ALGO

All problems which are generally solved using greedy algorithms satisfy TWO properties:

① Greedy choice property: A global optimum can be arrived at by selecting a local optimum.

② Optimal Substructure: An optimal solution to the problem contains an optimal solution to sub-problems. Problems whose optimal solution can be constructed efficiently from optimal solutions of its subproblems.

⊛ A Greedy Algo never reconsider its choices. Main diff from DP.

②

Self Algo Anal 8.1.22

Greedy ALGO's w/ Tim Kearns

What is Greedy ALGO?

An algorithm that constructs the solution to an optimization problem piece by piece through a sequence of choices that are:

- feasible = meets the requirements specified by the problem.
- locally optimal = makes the solution larger using some criteria ~~that~~
- irrevocable = do not change any previous partial sol'n. meaning that you aren't allowed to go back and change any previous partial sol'n. You can't go back and revisit things that you have already done.

A greedy algorithm can be used to

- yield an optimal sol'n for a problem.
- provide a fast approximation when other approaches are too inefficient. Greedy algorithms don't give an optimal sol'n but provide an approximation.

③ Self

Greedy w/ Tim Kearns

Apps of the Greedy Strategy

Optimal Solns $\rightarrow$ change making for normal coin
 $\rightarrow$ min spanning tree
 Huffman codes $\rightarrow$ single source shortest paths
 $\rightarrow$ some scheduling problems

Approximations $\rightarrow$ traveling sales person (TSP)

$\rightarrow$ knapsack
 $\rightarrow$ other combinatorial optimization problems

sinr
 $\rightarrow$ sinr, skrama, 2place, engellenen

BRANCH & BOUND w/ TIM KEARNS

BB = an approach to developing algo's for solving optimization prb's where you can create a "bound" on the objective function for the solns that occur in the subtree.

- As we search down the branches of the state space tree,
 - compute the bound (usually) by combining a partial evaluation of the func'n along with a computation of the best we can possibly do by continuing down the tree.
- if this bound is worse than what we have achieved with some actual sol'n (the optimal sol'n so far),
 - don't continue down that subtree since there is no point in evaluating possible solns along that branch - we say the subtree is pruned.
 - Example = Suppose want to find a max and have found a possible sol'n with value 100. if you know a subtree can have candidate solns whose value at most 90 then why is on!

(4)

Self

Algo Anal

8.1.22

DP w/Tim Kearns

Optimal substructure = ~~the~~ the larger problem contain the sol'n to smaller problem.

* Table Filling = We want initial conditions to make algo simple/avoid special code for working on the boundary.
 So make the table with an extra row and column.
 And initialize these entries with proper values.
For example in "robot in a grid" example fill the table's initial cond's with 0. Narely $F(i, j) = 0$ if $i=0$ or $j=0$
For example in knapsack fill table $F[i][j] = 0$ where $i=0$ or $j=0$

* Table dimension can add paramter vsa okade okade.
Ex-ple = in knapsack's recurrence relation

$$V(n, W) = \max \{ V(n-1, W), V(n-1, W-w_n) + v_n \}$$

2 paramter var table 2 bogethe okade

table'n base case wali upun value ile daleke in this
 ex-ple '0'

(1) Self [AT go Anand 12.1.22]

```
void top-sort (GRAPH *g, QUEUE *q)
```

```
{ put-queue(g, q)
```

```
  int src
```

```
  NODE *tmp
```

```
  while (!is-qu-empty(q))
```

```
  { src = dequeue(q);
```

```
    printf("job %d is done\n", src);
```

```
    tmp = g->array[src].head;
```

```
    g->indegree[src]--; // -1 yahtik dit -1 assignment
```

```
    while (tmp)
```

```
    { g->indegree[tmp->to-succ] --;
```

```
      if (g->indegree[tmp->to-succ] == 0)
```

```
        enqueue(q, tmp->to-succ);
```

```
      tmp = tmp->next;
```

```
    }
```

```
}
```

```
void put-queue (GRAPH *g, QUEUE *q)
```

```
{ int i;
```

```
  for (i = 0; i < g->numofnodes; i++)
```

```
    if (g->indegree[i] == 0)
```

```
      enqueue(q, i);
```

```
}
```

(*) NON-RECURSIVE Topological sort

② Self Algo Hand 12.1.22

void top-sort-recursive-completer (GRAPH *g)

```
{
    int src;
    src = find-indegree-zero(g);
    while (src > -1)
    {
        top-sort-recursive(g, src);
        src = find-indegree-zero(g);
    }
}
```

void top-sort-recursive (GRAPH *g, int src)

```
{
    NODE *t-p;
    g->visited[src] = 1;
    g->indegree[src]--; // assignment -1.
    printf("j %d is done\n");
    t-p = g->array[src].head;
    while (t-p)
    {
        g->indegree[t-p->kon-su-adi]--;
        if (g->indegree[t-p->kon-su-adi] == 0)
        {
            top-sort-recursive(g, t-p->kon-su-adi);
        }
        t-p = t-p->next;
    }
}
```

int find-indegree-zero (GRAPH *g)

```
{
    int i = 0;
    while ((i < g->num-of-nodes && g->indegree[i] != 0) || g->indegree[i] == -1)
    {
        i++;
    }
    if (i < g->num-of-nodes)
        return i;
    else
        return -1;
}
```

⊛ Top sort recursive!

③ Self Algo Arad 12.1.22

```
int solve-n-queen(int **table, int size, int current_row)
```

```
{ if (row == size)
```

```
{ printf("success\n");
```

```
print-table(table, size);
```

```
return 1;
```

```
}
```

```
int i; // represents columns!
```

```
for(i=0; i < size; i++)
```

```
{ printf("trying to put queen to (%d, %d) \n", current_row, i);
```

```
if (isSafe(table, current_row, i, size))
```

```
{ table[current_row][i] = 1;
```

```
if (solve-n-queen(table, size, current_row+1))
```

```
{ return 1; }
```

```
table[current_row][i] = 0; // backtrack.
```

```
}
```

```
} printf("no solution for row %d \n", current_row);
```

```
return 0;
```

```
}
```

```
int isSafe(int **table, int row, int col, int size)
```

```
{ int i, j, k;
```

```
for(i=0; i < row; i++)
```

```
if (table[i][col] == 1)
```

```
return 0;
```

```
for(i=0; i < col; i++)
```

```
if (table[row][i] == 1)
```

```
return 0;
```

```
for(i=row, j=col; i >= 0 && j >= 0; i--, j--)
```

```
if (table[i][j] == 1)
```

```
return 0;
```

```
for(i=row, j=col; i <= 0 && j < size; i--, j++)
```

```
if (table[i][j] == 1)
```

```
return 0;
```

```
return 1;
```

```
}
```

(4)

Algo Anal

```
int solve_hamiltonian(int **g, int size, int *path, int pose)
```

```
{
    if (pose == size)
    {
        if (g[path[pose]][path[pose-1]] == 1)
        {
            printf("success\n")
            print_path(path, size);
            return 1;
        }
        else
        {
            return 0;
        }
    }
}
```

```
int v; // represents vertex
```

```
for (v=0; v<size; v++)
```

```
{
    if (adding_safe(g, size, path, v, pose))
    {
        path[pose] = v;
        if (solve_hamiltonian(g, size, path, pose+1))
        {
            return 1;
        }
        path[pose] = -1; // backtrack.
    }
}
return 0;
}
```

```
int adding_safe(int **g, int size, int *path, int vertex, int pose)
```

```
{
    if (g[path[pose-1]][vertex] == 0)
        return 0;
}
```

```
int i;
for (i=0; i<pose; i++)
    if (path[i] == vertex)
        return 0;
}
```

```
return 1;
}
```